University Course Management System using MySQL

SQL Assignment – Joins, Aggregation, Functions, and Stored Procedures

PREPARED FOR

Mr. Mohammad Jahangir Alam(MJA) | Assistant Professor at DIU

PREPARED BY

MD Suzon Mia | 241-15-717 | Section: 66_H

This Report Presented in Partial Fulfillment of the course CSE311: Database Management System in the Computer Science and Engineering Department

DECLARATION

We hereby declare that this SQL assignment titled “University Course Management System using MySQL” has been completed by us under the supervision of Partha Dip Sarkar, Lecturer, Department of Computer Science and Engineering, Daffodil International University. We further declare that this work is our own and that neither this assignment nor any part of it has been submitted elsewhere for academic evaluation.

Submitted To:

Mr. Mohammad Jahangir Alam(MJA)
Assistant Professor
Department of Computer Science and Engineering
Daffodil International University

Submitted by:

Name :MD Suzon Mia
ID:241-15-717
Section: 66_H

Assignment Repository

https://github.com/mdsuzon-hub/cse_66/tree/master/assets/DBMS_Assignment

Department of Computer Science and Engineering
DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

Database Schema Overview

The database contains five related tables modelling a university academic system. All tables were created in MySQL with appropriate PRIMARY KEY and FOREIGN KEY constraints.

Table	Columns
Student	StudentID INT (PK), Name VARCHAR(100), Department VARCHAR(100), EnrollmentYear INT
Course	CourseID INT (PK), CourseName VARCHAR(100), Department VARCHAR(100), Credits INT
Enrollment	StudentID INT (FK), CourseID INT (FK), Semester VARCHAR(10), Grade FLOAT
Instructor	InstructorID INT (PK), Name VARCHAR(100), Department VARCHAR(100)
Teaches	InstructorID INT (FK), CourseID INT (FK), Semester VARCHAR(10)

Table Creation & Sample Data

All five tables are created with the SQL below. 20 rows of realistic sample data are inserted into each table.

```sql
-- Drop and recreate schema
DROP DATABASE IF EXISTS university_db;
CREATE DATABASE university_db;
USE university_db;

-- TABLE CREATION
CREATE TABLE Student (
    StudentID     INT PRIMARY KEY,
    Name          VARCHAR(100),
    Department    VARCHAR(100),
    EnrollmentYear INT
);

CREATE TABLE Course (
    CourseID    INT PRIMARY KEY,
    CourseName  VARCHAR(100),
    Department  VARCHAR(100),
    Credits     INT
);

CREATE TABLE Enrollment (
    StudentID  INT,
    CourseID   INT,
    Semester   VARCHAR(10),
    Grade      FLOAT,
    PRIMARY KEY (StudentID, CourseID, Semester),
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY (CourseID)  REFERENCES Course(CourseID)
);

CREATE TABLE Instructor (
    InstructorID INT PRIMARY KEY,
    Name         VARCHAR(100),
    Department   VARCHAR(100)
);
CREATE TABLE Teaches (
```

```sql
    InstructorID INT,
    CourseID     INT,
    Semester     VARCHAR(10),
    PRIMARY KEY (InstructorID, CourseID, Semester),
    FOREIGN KEY (InstructorID) REFERENCES Instructor(InstructorID),
    FOREIGN KEY (CourseID)     REFERENCES Course(CourseID)
);
-- SAMPLE DATA (20 rows per table)
-- Students
INSERT INTO Student VALUES
(1,  'আলিস রহমান',        'CSE', 2019),
(2,  'বাবু হোসেন',        'EEE', 2020),
(3,  'কারলা আক্তার',      'CSE', 2021),
(4,  'দাউদ ইসলাম',        'BBA', 2021),
(5,  'ইভা বেগম',          'CSE', 2022),
(6,  'ফাহিম উদ্দিন',       'EEE', 2022),
(7,  'গীতা শর্মা',        'BBA', 2021),
(8,  'হাসান রেজা',        'CSE', 2023),
(9,  'ইরিন সুলতানা',      'EEE', 2020),
(10, 'জালাল মিয়া',        'BBA', 2023),
(11, 'কামরুল হুদা',       'CSE', 2019),
(12, 'লিমা খাতুন',        'EEE', 2022),
(13, 'মাহবুব আলম',        'BBA', 2021),
(14, 'নাদিয়া ফারুক',      'CSE', 2022),
(15, 'ওমর ফারুক',         'EEE', 2023),
(16, 'পূজা রায়',          'BBA', 2020),
(17, 'কামার জামান',       'CSE', 2021),
(18, 'রিনা চৌধুরী',       'EEE', 2023),
(19, 'সাজিব মোল্লা',      'BBA', 2022),
(20, 'তানিয়া আবেদিন',     'CSE', 2020);


-- Courses
INSERT INTO Course VALUES
(101, 'Data Structures',        'CSE', 3),
(102, 'Algorithms',             'CSE', 3),
(103, 'Circuit Analysis',       'EEE', 3),
(104, 'Signals & Systems',      'EEE', 3),
(105, 'Marketing Management',   'BBA', 3),
(106, 'Financial Accounting',   'BBA', 3),
(107, 'Database Systems',       'CSE', 3),
(108, 'Operating Systems',      'CSE', 3),
(109, 'Digital Electronics',    'EEE', 3),
(110, 'Business Statistics',    'BBA', 3),
(111, 'Machine Learning',       'CSE', 3),
(112, 'Computer Networks',      'CSE', 3),
(113, 'Power Systems',          'EEE', 3),
(114, 'Managerial Economics',   'BBA', 3),
(115, 'Software Engineering',   'CSE', 3),
(116, 'Microprocessors',        'EEE', 3),
(117, 'Human Resource Mgmt',    'BBA', 3),
(118, 'Artificial Intelligence','CSE', 3),
(119, 'Control Systems',        'EEE', 3),
(120, 'Business Communication', 'BBA', 3);

-- Instructors
INSERT INTO Instructor VALUES
(201, 'Dr. Arifur Rahman',   'CSE'),
(202, 'Dr. Bilkis Jahan',    'EEE'),
(203, 'Dr. Chand Ali',       'BBA'),
(204, 'Prof. Delwar Hossain','CSE'),
(205, 'Prof. Esrat Jahan',   'EEE'),
```

```sql
(206, 'Dr. Farhana Begum',   'BBA'),
(207, 'Dr. Golam Kibria',    'CSE'),
(208, 'Prof. Hasina Parvin', 'EEE'),
(209, 'Dr. Imran Hasan',     'BBA'),
(210, 'Dr. Jasim Uddin',     'CSE'),
(211, 'Prof. Khaleda Akter', 'EEE'),
(212, 'Dr. Liton Miah',      'BBA'),
(213, 'Dr. Motiur Rahman',   'CSE'),
(214, 'Prof. Nasrin Islam',  'EEE'),
(215, 'Dr. Ohidul Islam',    'BBA'),
(216, 'Dr. Parvin Sultana',  'CSE'),
(217, 'Prof. Rafiq Ahmed',   'EEE'),
(218, 'Dr. Salma Khatun',    'BBA'),
(219, 'Dr. Toufiq Hasan',    'CSE'),
(220, 'Prof. Ulfat Ara',     'EEE');

-- Teaches
INSERT INTO Teaches VALUES
(201, 101, 'Spring2024'),
(201, 107, 'Fall2023'),
(202, 103, 'Spring2024'),
(202, 109, 'Fall2023'),
(203, 105, 'Spring2024'),
(203, 106, 'Fall2023'),
(204, 102, 'Spring2024'),
(204, 108, 'Fall2023'),
(205, 104, 'Spring2024'),
(205, 113, 'Fall2023'),
(206, 110, 'Spring2024'),
(206, 114, 'Fall2023'),
(207, 111, 'Spring2024'),
(207, 115, 'Fall2023'),
(208, 116, 'Spring2024'),
(208, 119, 'Fall2023'),
(209, 117, 'Spring2024'),
(209, 120, 'Fall2023'),
(210, 112, 'Spring2024'),
(210, 118, 'Spring2024');

-- Enrollments
INSERT INTO Enrollment VALUES
(1,  101, 'Spring2024', 3.7),
(2,  103, 'Spring2024', 3.2),
(3,  101, 'Spring2024', 3.9),
(4,  105, 'Spring2024', 2.8),
(5,  101, 'Spring2024', 3.5),
(6,  103, 'Spring2024', 2.5),
(7,  105, 'Spring2024', 3.1),
(8,  107, 'Spring2024', 3.8),
(9,  104, 'Spring2024', 3.0),
(10, 110, 'Spring2024', 2.3),
(11, 102, 'Spring2024', 1.8),
(12, 104, 'Spring2024', 3.6),
(13, 106, 'Spring2024', 2.9),
(14, 111, 'Spring2024', 3.4),
(15, 104, 'Spring2024', 2.7),
(16, 110, 'Spring2024', 3.3),
(17, 102, 'Spring2024', 3.6),
(18, 116, 'Spring2024', 2.1),
(19, 117, 'Spring2024', 3.0),
(20, 112, 'Spring2024', 3.7);
```

Sample Data Summary

20 rows inserted into each table: Student (IDs 1–20, departments CSE/EEE/BBA, enrollment years 2019–2023), Course (IDs 101–120, matching departments), Instructor (IDs 201–220), Teaches (Spring2024 & Fall2023 semesters), Enrollment (Spring2024 semester, grades 1.8–3.9).

1. Creating Database

Creating Database in MySQL (University Course Management System)

```
Run   Terminal   Help                                   Assignment

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

                                   mdsuz@DESKTOP-Q7062GA

                                   Windows 11 Home Single Language (25H2) x86_64
                                   11th Gen Intel(R) Core(TM) i5-1135G7 (8) @ 4.20 GHz
                                   X513EPN (1.0)
                                   11.62 GiB / 15.70 GiB (74%)
                                   314.02 GiB / 475.97 GiB (66%) - NTFS
                                   21.58 GiB / 400.39 GiB (5%) - NTFS
                                   355.47 GiB / 531.11 GiB (67%) - NTFS

                                   • • • • • • • •
                                   by tnka'

PS C:\Users\mdsuz\OneDrive\Desktop\DIU\Database Management System\Theory\Assignment> & "C:\xampp\mysql\bin\mysql.exe" -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 88
Server version: 10.4.32-MariaDB mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show databases;
+--------------------+
| Database           |
+--------------------+
| esp32_weather      |
| hall_meal_system   |
| information_schema |
| mysql              |
| performance_schema |
| phpmyadmin         |
| routine_db         |
| routinecompanion_db |
| smart_travel       |
| sportsbigdb        |
| sportsdb           |
| test               |
| test_db            |
| university_db      |
+--------------------+
14 rows in set (0.001 sec)

MariaDB [(none)]> DROP DATABASE IF EXISTS university_db;
Query OK, 5 rows affected (0.032 sec)

MariaDB [(none)]> CREATE DATABASE university_db;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> USE university_db;
Database changed
```

2. Creating Tables

Creating Tables for University Course Management System in MySQL

```
MariaDB [university_db]> CREATE TABLE Student (
    ->     StudentID     INT PRIMARY KEY,
    ->     Name          VARCHAR(100),
    ->     Department    VARCHAR(100),
    ->     EnrollmentYear INT
    -> );
Query OK, 0 rows affected (0.013 sec)

MariaDB [university_db]>
MariaDB [university_db]> CREATE TABLE Course (
    ->     CourseID    INT PRIMARY KEY,
    ->     CourseName  VARCHAR(100),
    ->     Department  VARCHAR(100),
    ->     Credits     INT
    -> );
Query OK, 0 rows affected (0.006 sec)

MariaDB [university_db]>
MariaDB [university_db]> CREATE TABLE Enrollment (
    ->     StudentID  INT,
    ->     CourseID   INT,
    ->     Semester   VARCHAR(10),
    ->     Grade      FLOAT,
    ->     PRIMARY KEY (StudentID, CourseID, Semester),
    ->     FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    ->     FOREIGN KEY (CourseID)  REFERENCES Course(CourseID)
    -> );
Query OK, 0 rows affected (0.024 sec)

MariaDB [university_db]>
MariaDB [university_db]> CREATE TABLE Instructor (
    ->     InstructorID INT PRIMARY KEY,
    ->     Name         VARCHAR(100),
    ->     Department   VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.008 sec)

MariaDB [university_db]>
MariaDB [university_db]> CREATE TABLE Teaches (
    ->     InstructorID INT,
    ->     CourseID     INT,
    ->     Semester     VARCHAR(10),
    ->     PRIMARY KEY (InstructorID, CourseID, Semester),
    ->     FOREIGN KEY (InstructorID) REFERENCES Instructor(InstructorID),
    ->     FOREIGN KEY (CourseID)     REFERENCES Course(CourseID)
    -> );
Query OK, 0 rows affected (0.028 sec)
```

3. Adding Data

Inserting Sample Data into Tables in MySQL Database

```
v   Go   Run   Terminal   Help                                   ←  →   Assignment

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\mdsuz\OneDrive\Desktop\DIU\Database Management System\Theory\Assignment> & "C:\xampp\mysql\bin\mysql.exe" -u root -p
MariaDB [university_db]> INSERT INTO Student VALUES
    -> (1,  [illegible],        'CSE', 2019),
    -> (2,  [illegible],        'EEE', 2020),
    -> (3,  [illegible],        'CSE', 2021),
    -> (4,  [illegible],        'BBA', 2021),
    -> (5,  [illegible],        'CSE', 2022),
    -> (6,  [illegible],        'EEE', 2022),
    -> (7,  [illegible],        'BBA', 2021),
    -> (8,  [illegible],        'CSE', 2023),
    -> (9,  [illegible],        'EEE', 2020),
    -> (10, [illegible],        'BBA', 2023),
    -> (11, [illegible],        'CSE', 2019),
    -> (12, [illegible],        'EEE', 2022),
    -> (13, [illegible],        'BBA', 2021),
    -> (14, [illegible],        'CSE', 2022),
    -> (15, [illegible],        'EEE', 2023),
    -> (16, [illegible],        'BBA', 2020),
    -> (17, [illegible],        'CSE', 2021),
    -> (18, [illegible],        'EEE', 2023),
    -> (19, [illegible],        'BBA', 2022),
    -> (20, [illegible],        'CSE', 2020);
Query OK, 20 rows affected (0.006 sec)
Records: 20  Duplicates: 0  Warnings: 0

MariaDB [university_db]> INSERT INTO Course VALUES
    -> (101, 'Data Structures',        'CSE', 3),
    -> (102, 'Algorithms',             'CSE', 3),
    -> (103, 'Circuit Analysis',       'EEE', 3),
    -> (104, 'Signals & Systems',      'EEE', 3),
    -> (105, 'Marketing Management',   'BBA', 3),
    -> (106, 'Financial Accounting',   'BBA', 3),
    -> (107, 'Database Systems',       'CSE', 3),
    -> (108, 'Operating Systems',      'CSE', 3),
    -> (109, 'Digital Electronics',    'EEE', 3),
    -> (110, 'Business Statistics',    'BBA', 3),
    -> (111, 'Machine Learning',       'CSE', 3),
    -> (112, 'Computer Networks',      'CSE', 3),
    -> (113, 'Power Systems',          'EEE', 3),
    -> (114, 'Managerial Economics',   'BBA', 3),
    -> (115, 'Software Engineering',   'CSE', 3),
    -> (116, 'Microprocessors',        'EEE', 3),
    -> (117, 'Human Resource Mgmt',    'BBA', 3),
    -> (118, 'Artificial Intelligence','CSE', 3),
    -> (119, 'Control Systems',        'EEE', 3),
    -> (120, 'Business Communication', 'BBA', 3);
Query OK, 20 rows affected (0.004 sec)
Records: 20  Duplicates: 0  Warnings: 0

MariaDB [university_db]> INSERT INTO Instructor VALUES
    -> (201, 'Dr. Arifur Rahman',   'CSE'),
    -> (202, 'Dr. Bilkis Jahan',    'EEE'),
    -> (203, 'Dr. Chand Ali',       'BBA'),
    -> (204, 'Prof. Delwar Hossain','CSE'),
    -> (205, 'Prof. Esrat Jahan',   'EEE'),
    -> (206, 'Dr. Farhana Begum',   'BBA'),
    -> (207, 'Dr. Golam Kibria',    'CSE'),
    -> (208, 'Prof. Hasina Parvin', 'EEE'),
    -> (209, 'Dr. Imran Hasan',     'BBA'),
    -> (210, 'Dr. Jasim Uddin',     'CSE'),
    -> (211, 'Prof. Khaleda Akter', 'EEE'),
    -> (212, 'Dr. Liton Miah',      'BBA'),
    -> (213, 'Dr. Motiur Rahman',   'CSE'),
    -> (214, 'Prof. Nasrin Islam',  'EEE'),
    -> (215, 'Dr. Ohidul Islam',    'BBA'),
    -> (216, 'Dr. Parvin Sultana',  'CSE'),
    -> (217, 'Prof. Rafiq Ahmed',   'EEE'),
    -> (218, 'Dr. Salma Khatun',    'BBA'),
    -> (219, 'Dr. Toufiq Hasan',    'CSE'),
    -> (220, 'Prof. Ulfat Ara',     'EEE');
Query OK, 20 rows affected (0.005 sec)
Records: 20  Duplicates: 0  Warnings: 0

MariaDB [university_db]> INSERT INTO Teaches VALUES
    -> (201, 101, 'Spring2024'),
    -> (201, 107, 'Fall2023'),
    -> (202, 103, 'Spring2024'),
    -> (202, 109, 'Fall2023'),
    -> (203, 105, 'Spring2024'),
    -> (203, 106, 'Fall2023'),
    -> (204, 102, 'Spring2024'),
    -> (204, 108, 'Fall2023'),
    -> (205, 104, 'Spring2024'),
```

Terminal Log:

```
C:\Users\mdsuz\OneDrive\Desktop\DIU\Database Management System\Theory\Assignment> &
"C:\xampp\mysql\bin\mysql.exe" -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 88
Server version: 10.4.32-MariaDB mariadb.org binary distribution
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show databases;
+--------------------+
| Database           |
+--------------------+
| esp32_weather      |
| hall_meal_system   |
| information_schema |
| mysql              |
| performance_schema |
| phpmyadmin         |
| routine_db         |
| routinecompanion_db |
| smart_travel       |
| sportsbigdb        |
| sportsdb           |
| test               |
| test_db            |
| university_db      |
+--------------------+
14 rows in set (0.001 sec)

MariaDB [(none)]> DROP DATABASE IF EXISTS university_db;
Query OK, 5 rows affected (0.032 sec)

MariaDB [(none)]> CREATE DATABASE university_db;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> USE university_db;
Database changed
MariaDB [university_db]> CREATE TABLE Student (
    ->     StudentID     INT PRIMARY KEY,
    ->     Name          VARCHAR(100),
    ->     Department    VARCHAR(100),
    ->     EnrollmentYear INT
    -> );
Query OK, 0 rows affected (0.013 sec)
```

```
MariaDB [university_db]>
MariaDB [university_db]> CREATE TABLE Course (
    ->     CourseID    INT PRIMARY KEY,
    ->     CourseName  VARCHAR(100),
    ->     Department  VARCHAR(100),
    ->     Credits     INT
    -> );
Query OK, 0 rows affected (0.006 sec)

MariaDB [university_db]>
MariaDB [university_db]> CREATE TABLE Enrollment (
    ->     StudentID  INT,
    ->     CourseID   INT,
    ->     Semester   VARCHAR(10),
    ->     Grade      FLOAT,
    ->     PRIMARY KEY (StudentID, CourseID, Semester),
    ->     FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    ->     FOREIGN KEY (CourseID)  REFERENCES Course(CourseID)
    -> );
Query OK, 0 rows affected (0.024 sec)

MariaDB [university_db]>
MariaDB [university_db]> CREATE TABLE Instructor (
    ->     InstructorID INT PRIMARY KEY,
    ->     Name         VARCHAR(100),
    ->     Department   VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.008 sec)

MariaDB [university_db]>
MariaDB [university_db]> CREATE TABLE Teaches (
    ->     InstructorID INT,
    ->     CourseID     INT,
    ->     Semester     VARCHAR(10),
    ->     PRIMARY KEY (InstructorID, CourseID, Semester),
    ->     FOREIGN KEY (InstructorID) REFERENCES Instructor(InstructorID),
    ->     FOREIGN KEY (CourseID)     REFERENCES Course(CourseID)
    -> );
Query OK, 0 rows affected (0.028 sec)

MariaDB [university_db]> INSERT INTO Student VALUES
    -> (1,  'আলিস রহমান',        'CSE', 2019),
    -> (2,  'বাবু হোসেন',        'EEE', 2020),
    -> (3,  'কারলা আক্তার',      'CSE', 2021),
```

```
    -> (4,  'দাউদ ইসলাম',        'BBA', 2021),
    -> (5,  'ইভা বেগম',          'CSE', 2022),
    -> (6,  'ফাহিম উদ্দিন',       'EEE', 2022),
    -> (7,  'গীতা শর্মা',        'BBA', 2021),
    -> (8,  'হাসান রেজা',        'CSE', 2023),
    -> (9,  'ইরিন সুলতানা',      'EEE', 2020),
    -> (10, 'জালাল মিয়া',        'BBA', 2023),
    -> (11, 'কামরুল হুদা',       'CSE', 2019),
    -> (12, 'লিমা খাতুন',        'EEE', 2022),
    -> (13, 'মাহবুব আলম',        'BBA', 2021),
    -> (14, 'নাদিয়া ফারুক',      'CSE', 2022),
    -> (15, 'ওমর ফারুক',         'EEE', 2023),
    -> (16, 'পূজা রায়',          'BBA', 2020),
    -> (17, 'কামার জামান',       'CSE', 2021),
    -> (18, 'রিনা চৌধুরী',       'EEE', 2023),
    -> (19, 'সাজিব মোল্লা',      'BBA', 2022),
    -> (20, 'তানিয়া আবেদিন',     'CSE', 2020);
Query OK, 20 rows affected (0.006 sec)
Records: 20  Duplicates: 0  Warnings: 0

MariaDB [university_db]> INSERT INTO Course VALUES
    -> (101, 'Data Structures',        'CSE', 3),
    -> (102, 'Algorithms',             'CSE', 3),
    -> (103, 'Circuit Analysis',       'EEE', 3),
    -> (104, 'Signals & Systems',      'EEE', 3),
    -> (105, 'Marketing Management',   'BBA', 3),
    -> (106, 'Financial Accounting',   'BBA', 3),
    -> (107, 'Database Systems',       'CSE', 3),
    -> (108, 'Operating Systems',      'CSE', 3),
    -> (109, 'Digital Electronics',    'EEE', 3),
    -> (110, 'Business Statistics',    'BBA', 3),
    -> (111, 'Machine Learning',       'CSE', 3),
    -> (112, 'Computer Networks',      'CSE', 3),
    -> (113, 'Power Systems',          'EEE', 3),
    -> (114, 'Managerial Economics',   'BBA', 3),
    -> (115, 'Software Engineering',   'CSE', 3),
    -> (116, 'Microprocessors',        'EEE', 3),
    -> (117, 'Human Resource Mgmt',    'BBA', 3),
    -> (118, 'Artificial Intelligence','CSE', 3),
    -> (119, 'Control Systems',        'EEE', 3),
    -> (120, 'Business Communication', 'BBA', 3);
Query OK, 20 rows affected (0.004 sec)
Records: 20  Duplicates: 0  Warnings: 0

MariaDB [university_db]> INSERT INTO Instructor VALUES
```

```
    -> (201, 'Dr. Arifur Rahman',   'CSE'),
    -> (202, 'Dr. Bilkis Jahan',    'EEE'),
    -> (203, 'Dr. Chand Ali',       'BBA'),
    -> (204, 'Prof. Delwar Hossain','CSE'),
    -> (205, 'Prof. Esrat Jahan',   'EEE'),
    -> (206, 'Dr. Farhana Begum',   'BBA'),
    -> (207, 'Dr. Golam Kibria',    'CSE'),
    -> (208, 'Prof. Hasina Parvin', 'EEE'),
    -> (209, 'Dr. Imran Hasan',     'BBA'),
    -> (210, 'Dr. Jasim Uddin',     'CSE'),
    -> (211, 'Prof. Khaleda Akter', 'EEE'),
    -> (212, 'Dr. Liton Miah',      'BBA'),
    -> (213, 'Dr. Motiur Rahman',   'CSE'),
    -> (214, 'Prof. Nasrin Islam',  'EEE'),
    -> (215, 'Dr. Ohidul Islam',    'BBA'),
    -> (216, 'Dr. Parvin Sultana',  'CSE'),
    -> (217, 'Prof. Rafiq Ahmed',   'EEE'),
    -> (218, 'Dr. Salma Khatun',    'BBA'),
    -> (219, 'Dr. Toufiq Hasan',    'CSE'),
    -> (220, 'Prof. Ulfat Ara',     'EEE');
Query OK, 20 rows affected (0.005 sec)
Records: 20  Duplicates: 0  Warnings: 0

MariaDB [university_db]> INSERT INTO Teaches VALUES
    -> (201, 101, 'Spring2024'),
    -> (201, 107, 'Fall2023'),
    -> (202, 103, 'Spring2024'),
    -> (202, 109, 'Fall2023'),
    -> (203, 105, 'Spring2024'),
    -> (203, 106, 'Fall2023'),
    -> (204, 102, 'Spring2024'),
    -> (204, 108, 'Fall2023'),
    -> (205, 104, 'Spring2024'),
    -> (205, 113, 'Fall2023'),
    -> (206, 110, 'Spring2024'),
    -> (206, 114, 'Fall2023'),
    -> (207, 111, 'Spring2024'),
    -> (207, 115, 'Fall2023'),
    -> (208, 116, 'Spring2024'),
    -> (208, 119, 'Fall2023'),
    -> (209, 117, 'Spring2024'),
    -> (209, 120, 'Fall2023'),
    -> (210, 112, 'Spring2024'),
    -> (210, 118, 'Spring2024');
Query OK, 20 rows affected (0.008 sec)
```

```
Records: 20  Duplicates: 0  Warnings: 0

MariaDB [university_db]> INSERT INTO Enrollment VALUES
    -> (1,  101, 'Spring2024', 3.7),
    -> (2,  103, 'Spring2024', 3.2),
    -> (3,  101, 'Spring2024', 3.9),
    -> (4,  105, 'Spring2024', 2.8),
    -> (5,  101, 'Spring2024', 3.5),
    -> (6,  103, 'Spring2024', 2.5),
    -> (7,  105, 'Spring2024', 3.1),
    -> (8,  107, 'Spring2024', 3.8),
    -> (9,  104, 'Spring2024', 3.0),
    -> (10, 110, 'Spring2024', 2.3),
    -> (11, 102, 'Spring2024', 1.8),
    -> (12, 104, 'Spring2024', 3.6),
    -> (13, 106, 'Spring2024', 2.9),
    -> (14, 111, 'Spring2024', 3.4),
    -> (15, 104, 'Spring2024', 2.7),
    -> (16, 110, 'Spring2024', 3.3),
    -> (17, 102, 'Spring2024', 3.6),
    -> (18, 116, 'Spring2024', 2.1),
    -> (19, 117, 'Spring2024', 3.0),
    -> (20, 112, 'Spring2024', 3.7);
Query OK, 20 rows affected (0.008 sec)
Records: 20  Duplicates: 0  Warnings: 0
```

Task 1: SQL JOINs

Objective
List all students along with the courses they are enrolled in and the names of the instructors teaching those courses during the Spring2024 semester.

Query:

```
SELECT
    s.StudentID,
    s.Name        AS StudentName,
    c.CourseName,
    i.Name        AS InstructorName
FROM Student s
JOIN Enrollment e  ON s.StudentID   = e.StudentID   AND e.Semester = 'Spring2024'
JOIN Course     c  ON e.CourseID    = c.CourseID
JOIN Teaches    t  ON c.CourseID    = t.CourseID     AND t.Semester = 'Spring2024'
JOIN Instructor i  ON t.InstructorID = i.InstructorID
ORDER BY s.StudentID;
```

The query uses four INNER JOINs to connect all five tables. The Semester = 'Spring2024' condition is applied on both Enrollment and Teaches to ensure only courses active in that semester are returned. This gives each student their course name and the instructor teaching it.

Output:

```
MariaDB [university_db]> SELECT
    ->     s.StudentID,
    ->     s.Name        AS StudentName,
    ->     c.CourseName,
    ->     i.Name        AS InstructorName
    -> FROM Student s
    -> JOIN Enrollment e  ON s.StudentID   = e.StudentID   AND e.Semester = 'Spring2024'
    -> JOIN Course     c  ON e.CourseID    = c.CourseID
    -> JOIN Teaches    t  ON c.CourseID    = t.CourseID     AND t.Semester = 'Spring2024'
    -> JOIN Instructor i  ON t.InstructorID = i.InstructorID
    -> ORDER BY s.StudentID;
+-----------+-------------------------------------+----------------------+----------------------+
| StudentID | StudentName                         | CourseName           | InstructorName       |
+-----------+-------------------------------------+----------------------+----------------------+
|         1 | আলিস রহমান                          | Data Structures      | Dr. Arifur Rahman    |
|         2 | বাবু হোসেন                          | Circuit Analysis     | Dr. Bilkis Jahan     |
|         3 | কারলা আক্তার                        | Data Structures      | Dr. Arifur Rahman    |
|         4 | দাউদ ইসলাম                          | Marketing Management | Dr. Chand Ali        |
|         5 | ইভা বেগম                            | Data Structures      | Dr. Arifur Rahman    |
|         6 | ফাহিম উদ্দিন                        | Circuit Analysis     | Dr. Bilkis Jahan     |
|         7 | গীতা শর্মা                          | Marketing Management | Dr. Chand Ali        |
|         9 | ইরিন সুলতানা                        | Signals & Systems    | Prof. Esrat Jahan    |
|        10 | জালাল মিয়া                         | Business Statistics  | Dr. Farhana Begum    |
|        11 | কামরুল হুদা                         | Algorithms           | Prof. Delwar Hossain |
|        12 | লিমা খাতুন                          | Signals & Systems    | Prof. Esrat Jahan    |
|        14 | নাদিয়া ফারুক                       | Machine Learning     | Dr. Golam Kibria     |
|        15 | ওমর ফারুক                           | Signals & Systems    | Prof. Esrat Jahan    |
|        16 | পূজা রায়                           | Business Statistics  | Dr. Farhana Begum    |
|        17 | কামার জামান                         | Algorithms           | Prof. Delwar Hossain |
|        18 | রিনা চৌধুরী                         | Microprocessors      | Prof. Hasina Parvin  |
|        19 | সাজিব মোল্লা                        | Human Resource Mgmt  | Dr. Imran Hasan      |
|        20 | তানিয়া আবেদিন                      | Computer Networks    | Dr. Jasim Uddin      |
+-----------+-------------------------------------+----------------------+----------------------+
18 rows in set (0.001 sec)
```

Task 2: Aggregation with Condition

Objective
Calculate the average grade for each course, considering only students who enrolled after 2020 (EnrollmentYear > 2020).

Query:

```
SELECT
    c.CourseID,
    c.CourseName,
    ROUND(AVG(e.Grade), 2) AS AverageGrade
FROM Enrollment e
JOIN Course  c ON e.CourseID  = c.CourseID
JOIN Student s ON e.StudentID = s.StudentID
WHERE s.EnrollmentYear > 2020
GROUP BY c.CourseID, c.CourseName
ORDER BY c.CourseID;
```

The WHERE clause filters to only students who enrolled after 2020. The GROUP BY clause groups results by course, and AVG(e.Grade) calculates the average grade within each group. ROUND(..., 2) ensures two decimal places in the output.

Output:

```
MariaDB [university_db]> SELECT
    ->     c.CourseID,
    ->     c.CourseName,
    ->     ROUND(AVG(e.Grade), 2) AS AverageGrade
    -> FROM Enrollment e
    -> JOIN Course  c ON e.CourseID  = c.CourseID
    -> JOIN Student s ON e.StudentID = s.StudentID
    -> WHERE s.EnrollmentYear > 2020
    -> GROUP BY c.CourseID, c.CourseName
    -> ORDER BY c.CourseID;
+----------+----------------------+--------------+
| CourseID | CourseName           | AverageGrade |
+----------+----------------------+--------------+
|      101 | Data Structures      |         3.70 |
|      102 | Algorithms           |         3.60 |
|      103 | Circuit Analysis     |         2.50 |
|      104 | Signals & Systems    |         3.15 |
|      105 | Marketing Management |         2.95 |
|      106 | Financial Accounting |         2.90 |
|      107 | Database Systems     |         3.80 |
|      110 | Business Statistics  |         2.30 |
|      111 | Machine Learning     |         3.40 |
|      116 | Microprocessors      |         2.10 |
|      117 | Human Resource Mgmt  |         3.00 |
+----------+----------------------+--------------+
11 rows in set (0.001 sec)
```

Task 3: User-Defined Function — calculateGPA

Objective
Create a MySQL function called calculateGPA(student_id INT) that returns the average GPA of the student from the Enrollment table.

Function Definition:

```
DROP FUNCTION IF EXISTS calculateGPA;

DELIMITER $$
CREATE FUNCTION calculateGPA(student_id INT)
RETURNS DECIMAL(4,2)
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE gpa DECIMAL(4,2);
    SELECT ROUND(AVG(Grade), 2) INTO gpa
    FROM Enrollment
    WHERE StudentID = student_id;
    RETURN IFNULL(gpa, 0.00);
END$$
DELIMITER ;
```

Test Query:

```
SELECT StudentID, Name, calculateGPA(StudentID) AS GPA
FROM Student
ORDER BY StudentID;
```

The function accepts a StudentID, queries the Enrollment table for that student's grades, computes the average with ROUND(..., 2), and returns it. IFNULL handles students with no enrollments by returning 0.00. READS SQL DATA and DETERMINISTIC are required attributes for MySQL functions that read from tables.

Output:

```
MariaDB [university_db]> DELIMITER $$
MariaDB [university_db]> CREATE FUNCTION calculateGPA(student_id INT)
    -> RETURNS DECIMAL(4,2)
    -> DETERMINISTIC
    -> READS SQL DATA
    -> BEGIN
    ->     DECLARE gpa DECIMAL(4,2);
    ->     SELECT ROUND(AVG(Grade), 2) INTO gpa
    ->     FROM Enrollment
    ->     WHERE StudentID = student_id;
    ->     RETURN IFNULL(gpa, 0.00);
    -> END$$
Query OK, 0 rows affected (0.005 sec)

MariaDB [university_db]> DELIMITER ;
MariaDB [university_db]> SELECT StudentID, Name, calculateGPA(StudentID) AS GPA
    -> FROM Student
    -> ORDER BY StudentID;
+-----------+-------------------------------------+------+
| StudentID | Name                                | GPA  |
+-----------+-------------------------------------+------+
|         1 | আলিস রহমান                          | 3.70 |
|         2 | বাবু হোসেন                          | 3.20 |
|         3 | কারলা আক্তার                        | 3.90 |
|         4 | দাউদ ইসলাম                          | 2.80 |
|         5 | ইভা বেগম                            | 3.50 |
|         6 | ফাহিম উদ্দিন                        | 2.50 |
|         7 | গীতা শর্মা                          | 3.10 |
|         8 | হাসান রেজা                          | 3.80 |
|         9 | ইরিন সুলতানা                        | 3.00 |
|        10 | জালাল মিয়া                         | 2.30 |
|        11 | কামরুল হুদা                         | 1.80 |
|        12 | লিমা খাতুন                          | 3.60 |
|        13 | মাহবুব আলম                          | 2.90 |
|        14 | নাদিয়া ফারুক                       | 3.40 |
|        15 | ওমর ফারুক                           | 2.70 |
|        16 | পূজা রায়                           | 3.30 |
|        17 | কামার জামান                         | 3.60 |
|        18 | রিনা চৌধুরী                         | 2.10 |
|        19 | সাজিব মোল্লা                        | 3.00 |
|        20 | তানিয়া আবেদিন                      | 3.70 |
+-----------+-------------------------------------+------+
20 rows in set (0.002 sec)
```

Task 4: Conditional Output with CASE WHEN

Objective
Using calculateGPA(), display each student's name and performance category: Excellent (>3.5), Good (3.0–3.5), Average (2.0–3.0), Poor (<2.0).

Test Query:

```
SELECT
    s.StudentID,
    s.Name,
    calculateGPA(s.StudentID) AS GPA,
    CASE
        WHEN calculateGPA(s.StudentID) > 3.5
THEN 'Excellent'
        WHEN calculateGPA(s.StudentID) >= 3.0 AND calculateGPA(s.StudentID) <= 3.5
THEN 'Good'
        WHEN calculateGPA(s.StudentID) >= 2.0 AND calculateGPA(s.StudentID) < 3.0
THEN 'Average'
        ELSE 'Poor'
    END AS PerformanceCategory
FROM Student s
ORDER BY s.StudentID;
```

The function accepts a StudentID, queries the Enrollment table for that student's grades, computes the average with ROUND(..., 2), and returns it. IFNULL handles students with no enrollments by returning 0.00. READS SQL DATA and DETERMINISTIC are required attributes for MySQL functions that read from tables.

Output:

```
MariaDB [university_db]> SELECT
    ->     s.StudentID,
    ->     s.Name,
    ->     calculateGPA(s.StudentID) AS GPA,
    ->     CASE
    ->         WHEN calculateGPA(s.StudentID) > 3.5                                    THEN 'Excellent'
    ->         WHEN calculateGPA(s.StudentID) >= 3.0 AND calculateGPA(s.StudentID) <= 3.5 THEN 'Good'
    ->         WHEN calculateGPA(s.StudentID) >= 2.0 AND calculateGPA(s.StudentID) < 3.0  THEN 'Average'
    ->         ELSE 'Poor'
    ->     END AS PerformanceCategory
    -> FROM Student s
    -> ORDER BY s.StudentID;
+-----------+-------------------------------------+------+---------------------+
| StudentID | Name                                | GPA  | PerformanceCategory |
+-----------+-------------------------------------+------+---------------------+
|         1 | আলিস রহমান                          | 3.70 | Excellent           |
|         2 | বাবু হোসেন                          | 3.20 | Good                |
|         3 | কারলা আক্তার                        | 3.90 | Excellent           |
|         4 | দাউদ ইসলাম                          | 2.80 | Average             |
|         5 | ইভা বেগম                            | 3.50 | Good                |
|         6 | ফাহিম উদ্দিন                        | 2.50 | Average             |
|         7 | গীতা শর্মা                          | 3.10 | Good                |
|         8 | হাসান রেজা                          | 3.80 | Excellent           |
|         9 | ইরিন সুলতানা                        | 3.00 | Good                |
|        10 | জালাল মিয়া                         | 2.30 | Average             |
|        11 | কামরুল হুদা                         | 1.80 | Poor                |
|        12 | লিমা খাতুন                          | 3.60 | Excellent           |
|        13 | মাহবুব আলম                          | 2.90 | Average             |
|        14 | নাদিয়া ফারুক                       | 3.40 | Good                |
|        15 | ওমর ফারুক                           | 2.70 | Average             |
|        16 | পূজা রায়                           | 3.30 | Good                |
|        17 | কামার জামান                         | 3.60 | Excellent           |
|        18 | রিনা চৌধুরী                         | 2.10 | Average             |
|        19 | সাজিব মোল্লা                        | 3.00 | Good                |
|        20 | তানিয়া আবেদিন                      | 3.70 | Excellent           |
+-----------+-------------------------------------+------+---------------------+
20 rows in set (0.003 sec)
```

Task 5: Stored Procedure with Cursor and Loop

Objective
Create a stored procedure that accepts a department name, loops through all students in that department, and displays their name and GPA using calculateGPA().

Stored Procedure

```
DROP PROCEDURE IF EXISTS GetDepartmentGPA;

DELIMITER $$
CREATE PROCEDURE GetDepartmentGPA(IN dept_name VARCHAR(100))
BEGIN
    DECLARE done      INT DEFAULT FALSE;
    DECLARE stu_id    INT;
    DECLARE stu_name  VARCHAR(100);
    DECLARE stu_gpa   DECIMAL(4,2);

    DECLARE cur CURSOR FOR
        SELECT StudentID, Name
        FROM Student
        WHERE Department = dept_name;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN cur;

    read_loop: LOOP
        FETCH cur INTO stu_id, stu_name;
        IF done THEN
            LEAVE read_loop;
        END IF;
        SET stu_gpa = calculateGPA(stu_id);
        SELECT stu_id AS StudentID, stu_name AS Name, stu_gpa AS GPA;
    END LOOP;

    CLOSE cur;
END$$
DELIMITER ;
```

Execute the procedure

```
CALL GetDepartmentGPA('CSE');
```

The procedure declares a CURSOR that selects all students in the given department. A CONTINUE HANDLER sets done = TRUE when no more rows are available. The LOOP repeatedly FETCHes a row, calls calculateGPA() for that student, and SELECTs the result until done is TRUE. The cursor is then CLOSED.

Output:

```
MariaDB [university_db]> DELIMITER $$
MariaDB [university_db]> CREATE PROCEDURE GetDepartmentGPA(IN dept_name VARCHAR(100))
    -> BEGIN
    ->     DECLARE done      INT DEFAULT FALSE;
    ->     DECLARE stu_id    INT;
    ->     DECLARE stu_name  VARCHAR(100);
    ->     DECLARE stu_gpa   DECIMAL(4,2);
    ->
```

```
    ->     DECLARE cur CURSOR FOR
    ->         SELECT StudentID, Name
    ->         FROM Student
    ->         WHERE Department = dept_name;
    ->
    ->     DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
    ->
    ->     OPEN cur;
    ->
    ->     read_loop: LOOP
    ->         FETCH cur INTO stu_id, stu_name;
    ->         IF done THEN
    ->             LEAVE read_loop;
    ->         END IF;
    ->         SET stu_gpa = calculateGPA(stu_id);
    ->         SELECT stu_id AS StudentID, stu_name AS Name, stu_gpa AS GPA;
    ->     END LOOP;
    ->
    ->     CLOSE cur;
    -> END$$
Query OK, 0 rows affected (0.007 sec)

MariaDB [university_db]> DELIMITER ;
MariaDB [university_db]> CALL GetDepartmentGPA('CSE');
+-----------+------------------------------+------+
| StudentID | Name                         | GPA  |
+-----------+------------------------------+------+
|         1 | আলিস রহমান                  | 3.70 |
+-----------+------------------------------+------+
1 row in set (0.001 sec)

+-----------+-----------------------------------+------+
| StudentID | Name                              | GPA  |
+-----------+-----------------------------------+------+
|         3 | কারলা আক্তার                     | 3.90 |
+-----------+-----------------------------------+------+
1 row in set (0.007 sec)

+-----------+------------------------+------+
| StudentID | Name                   | GPA  |
+-----------+------------------------+------+
|         5 | ইভা বেগম               | 3.50 |
+-----------+------------------------+------+
1 row in set (0.009 sec)

+-----------+------------------------------+------+
| StudentID | Name                         | GPA  |
+-----------+------------------------------+------+
|         8 | হাসান রেজা                  | 3.80 |
+-----------+------------------------------+------+
1 row in set (0.010 sec)

+-----------+--------------------------------+------+
| StudentID | Name                           | GPA  |
+-----------+--------------------------------+------+
|        11 | কামরুল হুদা                    | 1.80 |
+-----------+--------------------------------+------+
1 row in set (0.011 sec)

+-----------+-----------------------------------+------+
| StudentID | Name                              | GPA  |
+-----------+-----------------------------------+------+
```

```
|        14 | নাদিয়া ফারুক                        | 3.40 |
+-----------+-------------------------------------+------+
1 row in set (0.014 sec)

+-----------+--------------------------------+------+
| StudentID | Name                           | GPA  |
+-----------+--------------------------------+------+
|        17 | কামার জামান                    | 3.60 |
+-----------+--------------------------------+------+
1 row in set (0.017 sec)

+-----------+-------------------------------------+------+
| StudentID | Name                                | GPA  |
+-----------+-------------------------------------+------+
|        20 | তানিয়া আবেদিন                       | 3.70 |
+-----------+-------------------------------------+------+
1 row in set (0.019 sec)

Query OK, 8 rows affected (0.022 sec)
```

Bonus Task: Students Never Enrolled

Objective
Identify students who have never enrolled in any course.

Query:

```
INSERT INTO Student VALUES
(21, 'রাহিম উদ্দিন', 'CSE', 2026),
(22, 'শিলা আক্তার', 'EEE', 2026),
(23, 'তানভীর হাসান', 'BBA', 2026);

SELECT s.StudentID, s.Name, s.Department
FROM Student s
LEFT JOIN Enrollment e ON s.StudentID = e.StudentID
WHERE e.StudentID IS NULL;
```

A LEFT JOIN keeps all students even if they have no matching row in Enrollment. The WHERE e.StudentID IS NULL condition filters to only those students for whom no enrollment record was found — i.e., students who never enrolled in any course.

Output:

```
MariaDB [university_db]> INSERT INTO Student VALUES
    -> (21, 'রাহিম উদ্দিন', 'CSE', 2026),
    -> (22, 'শিলা আক্তার', 'EEE', 2026),
    -> (23, 'তানভীর হাসান', 'BBA', 2026);
Query OK, 3 rows affected (0.005 sec)
Records: 3  Duplicates: 0  Warnings: 0

MariaDB [university_db]> SELECT s.StudentID, s.Name, s.Department
    -> FROM Student s
    -> LEFT JOIN Enrollment e ON s.StudentID = e.StudentID
    -> WHERE e.StudentID IS NULL;
+-----------+-----------------------------------+------------+
| StudentID | Name                              | Department |
+-----------+-----------------------------------+------------+
|        21 | রাহিম উদ্দিন                      | CSE        |
|        22 | শিলা আক্তার                       | EEE        |
|        23 | তানভীর হাসান                      | BBA        |
+-----------+-----------------------------------+------------+
3 rows in set (0.001 sec)
```

References:

MySQL Documentation | https://dev.mysql.com/doc/ | MariaDB Official Documentation | https://mariadb.com/kb/en/documentation/ | W3Schools SQL Tutorial | https://www.w3schools.com/sql/ | GitHub Repository (Assignment Code) | https://github.com/mdsuzon-hub/cse_66/blob/master/DBMS_Assignment | Course Lecture Notes (Database Management System, CSE, Daffodil International University) | Internal Course Material

— End of Report —

MD SUZON MIA | ID: 241-15-717 | Section: 66_H
Daffodil International University — Compiler Design Laboratory